

Results

Fitz Koch

2026-04-03

Setup

```
import sys
sys.path.insert(0, "src")
```

Imports

```
## std lib
import importlib
from pathlib import Path
from itertools import product

## 3rd p lib
from IPython.display import display
import numpy as np
import matplotlib.pyplot as plt

# project code
import src.viz as viz
import src.constants as c

importlib.reload(viz)
from src.viz import viz_sim
```

Results

Currently, I've only run the 'restricted' breeding strategy with the 'name' sensing information. We don't get quite as good results as they do in the paper – they get down to **1.33** for name based and restricted breeding, but we only down to about **1.8** or so. This is almost certainly due to some bug, incorrect implementation, or unexpected side-effect or detail of the `lil-gp` paradigm, which I know basically nothing about.

Still, we can certainly see the results in an interesting fashion.

Animations

The meat of it.

```
base_path = Path("position_data")
terminals = ["Base", "Deitic", "Name"]
strategies = ["Clone", "Free", "Restricted"]
id = "zero-vec-update"

for terminal, strategy in product(terminals, strategies):
    path = base_path / terminal / strategy / f"{id}.npy"
    data = np.load(path)
    display(viz_sim(data[0], title=f"{terminal}, {strategy}: First Gen"))
    display(viz_sim(data[-1], title=f"{terminal}, {strategy}: Final Gen"))
    print(f"simulated {terminal}, {strategy}")
```

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

simulated Base, Clone

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

simulated Base, Free

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

simulated Base, Restricted

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

simulated Deitic, Clone

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

simulated Deitic, Free

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

simulated Deitic, Restricted

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

simulated Name, Clone

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

simulated Name, Free

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

simulated Name, Restricted

Results

```
def plot_loss_curves(clone_data, free_data, restricted_data, title):
    breeding_strategies = ["Clone", "Free", "Restricted"]
    data_list = [clone_data, free_data, restricted_data]
    colors = {"Clone": "steelblue", "Free": "darkorange", "Restricted": "seagreen"}
    n = len(clone_data["avg_loss"])
    fig, axes = plt.subplots(nrows=1, ncols=2)
    avg_ax, best_ax = axes

    for strategy, data in zip(breeding_strategies, data_list):
        avg_ax.plot(range(n), data["avg_loss"], label=strategy, color=colors[strategy])
        best_ax.plot(
            range(n), data["best_loss"], label=strategy, color=colors[strategy]
        )

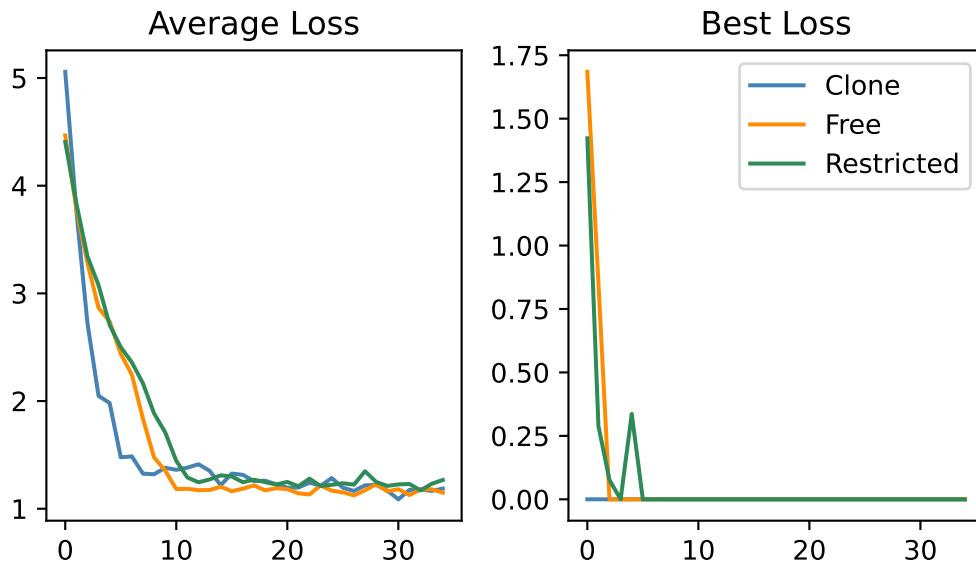
    best_ax.legend()
    avg_ax.set_title("Average Loss")
    best_ax.set_title("Best Loss")
    plt.suptitle(title)
    plt.tight_layout()
    return fig
```

```
sensing_terminals = ["Name", "Deitic", "Base"]
id = "zero-vec-update"
path_end = "_checkpoint.npz"

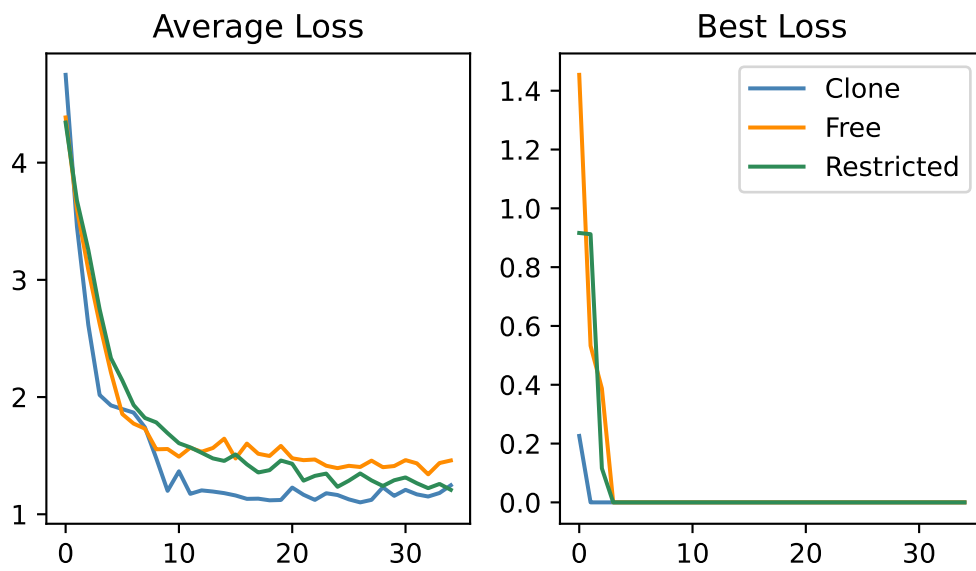
for terminal in sensing_terminals:
    clone_data = np.load(Path("results") / terminal / "Clone" / f"{id}-{path_end}")
    free_data = np.load(Path("results") / terminal / "Free" / f"{id}-{path_end}")
    restricted_data = np.load(
        Path("results") / terminal / "Restricted" / f"{id}-{path_end}"
    )

    plot_loss_curves(clone_data, free_data, restricted_data, title=terminal)
```

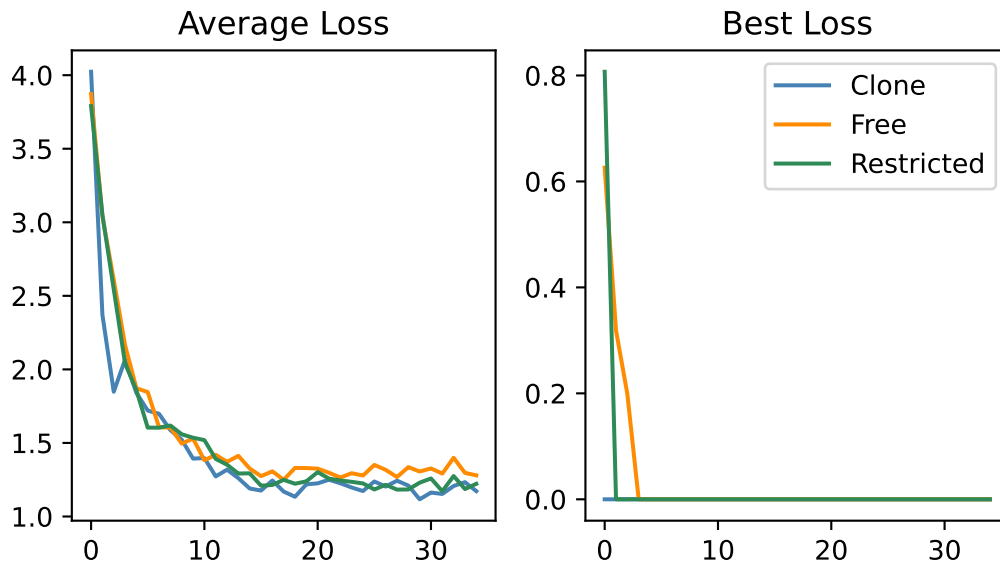
Name



Deitic



Base



The results definitely don't agree with those in the published paper – lions catch much more frequently, it seems, yet the average loss does not go as low. In addition, there is not nearly as much differentiation between the different breeding strategies and terminals as in their results. I suspect this is something to do with the details of the **name** terminals, though I'm not totally sure exactly what it is, and Claude has not been able to figure it out either.